

Trade Zero Coupon Bond

Technical Design Document

Document Type	High-level technical design
Design Focus	Ask Limit Order architecture, transaction model, parameters, datum, redeemers, and transaction constraints
Covered Flows	Create, Update/Cancel, Buy

1. Purpose

This document consolidates the on-chain technical design for Ask Limit Orders in the Trade Zero Coupon Bond module. It describes the selected design approach, the supported transaction flows, and the main validation and data structures required for implementation.

2. Functional Scope

- Create an Ask Limit Order by depositing PT into an Ask Limit UTxO.
- Update or cancel an existing Ask Limit Order.
- Buy from an Ask Limit Order, including partial continuation when applicable.
- Route owner proceeds and protocol fees correctly during order execution.
- Define the high-level parameter, datum, redeemer, and transaction architecture for the selected on-chain design.

3. User Stories

- As a seller, I want to create an Ask Limit Order so that I can deposit PT and offer it for sale at a requested yield.
- As a seller, I want to update or cancel my Ask Limit Order so that I can change the order or withdraw it when needed.
- As a buyer, I want to buy PT from an Ask Limit Order so that I can execute against available liquidity.
- As the protocol, I want fees to be routed to the configured fee address so that order execution follows the configured fee model.

4. Success Metrics

Testnet implementation supports the three Ask Limit Order transaction flows: Create Update/Cancel, and Buy.

5. Design Overview

The on-chain design reuses the Bond DEX design pattern for Ask Limit Orders. Within the scope of this module, the Ask Limit Script holds the deposited PT and represents the seller's limit order.

High-level state machine:

- Create: create an active Ask Limit Order.
- Update/Redeem: modify an existing active order.
- Buy: spend an existing order and optionally continue the remaining Ask Limit Order.
- Cancel: close an existing order.

6. Design Options Reviewed

The original design reviewed three options for owner validation and UTxO identification.

Option	Description	Pros	Cons
1	Validate owner by payment key and do not use FT/NFT to identify the UTxO. This is the preferred option.	Simple because it follows the existing Bond DEX implementation. No additional minting validator is required.	Changing the owner requires spending the Order UTxO. Script owners are not supported because the signer check cannot validate a script owner. Inputs must validate datum correctness because no FT/NFT authenticates the UTxO.
2	Validate owner by payment key and use FT/NFT to identify the UTxO.	Ensures the validity of the created UTxO and makes FT/NFT lifecycle clearer during create, match, and cancel.	Adds minting logic and increases on-chain complexity and cost.
3	Validate owner by Owner NFT and use FT/NFT to identify the UTxO.	Ensures the validity of the created UTxO and allows owner transfer without spending the Order UTxO.	Adds minting logic and increases complexity. Introduces a redeem requirement for matched assets instead of direct receipt during buy.

7. Selected Design

The selected design is Option 1: validate the owner by payment key and do not use FT/NFT to identify the UTxO.

- The owner of the order is determined by the payment key stored in the Ask Limit datum.
- No NFT is minted and inserted into the UTxO during order creation.
- The UTxO is not validated at create time by a script; validation is performed when the order is spent for matching or cancellation.

8. Protocol Parameters and Datum

Selected design protocol configuration:

```
type ProtocolConfig {
    escrow: EscrowConfig,
    exchange: ExchangeConfigLimit,
}

type EscrowConfig = List<PolicyId>

type ExchangeConfigLimit {
    fee_addr: Address,
    buyer_fee: Int,    // basisPoint
    seller_fee: Int,   // basisPoint
    tx_buy_ttl: Option<Int>,
}
```

Definitions:

- escrow: list of supported PT policies.
- Exchange:
 - fee_addr: address that receives protocol fees.
 - buyer_fee: fee charged to the buyer when matching an order.
 - seller_fee: fee charged to the seller when matching an order.
 - tx_buy_ttl: validity window of the Buy transaction in slots; default is 360 when null.

Ask Limit Order datum:

```
type AskLimitOrderDatum {
    owner_vk: PublicKeyHash,
    requested_yield: Int, // basisPoint
}

type PublicKeyHash = Hash<Blake2b_224, VerificationKey>
```

- owner_vk: payment key hash of the owner wallet address. This signature is required for order update or cancel.
- requested_yield: yield requested by the order.

9. Redeemers and Validation Model

```
type SpendAction {
    Update,
    Buy((Int, Int, (TupleAsset, Int), Option<Int>)),
}

type WithdrawAction {
    BuyMulti
}
```

Validation behavior:

- Update is used for order update or cancel and requires the transaction to be signed by owner_vk in the datum.
- Buy is used to match the order and includes exchange_fee, seller_receive, offer (PT and quantity sold), and cont_idx for partial continuation.
- Spend validation of Buy requires successful execution of the Ask Limit Script Withdraw script with redeemer BuyMulti.
- Withdraw validation checks consistency between the Withdraw redeemer and the Spend redeemers, and validates owner output, fee output, and continuation order output when present.

10. Transaction Architecture

10.1 Create Ask Limit Order

- Signatories: user wallet.
- Script invoked: none.
- Reference inputs: none.
- Inputs: user UTxO.
- Outputs: Ask Limit UTxO(s) holding Ask Limit configuration and deposited PT.
- Constraint: none, because no script is invoked at creation time.

10.2 Update or Cancel Ask Limit Order

- Signatories: user wallet.
- Script invoked: Ask Limit Script Spend.
- Reference inputs: none.
- Inputs: Ask Limit UTxO.
- Outputs: Ask Limit UTxO (optional).
- Constraint: ask_in_datum.owner_vk must be present in the transaction.

10.3 Buy Ask Limit Order

- Signatories: user wallet.
- Scripts invoked: Ask Limit Script Spend and Ask Limit Script Withdraw.
- Reference inputs: Fixed Lending Pool UTxO.
- Inputs: Ask Limit UTxO.

- Outputs: Ask Limit UTxO (optional), Owner UTxO, Fee UTxO.

Buy transaction output constraints:

Output	Field	Constraint
Ask Limit Out	Address	Must equal the address of the Ask Limit input UTxO.
Ask Limit Out	Value	<code>ask_out_val.pt > 0</code> .
Ask Limit Out	Datum	Unchanged.
Owner UTxO	Address	<code>owner_vk</code> plus the stake credential of the Ask Limit input UTxO.
Owner UTxO	Value	<code>origin_token</code> .
Fee UTxO	Address	<code>protocol_config.fee_address</code> .
		<code>origin_token</code> ; charged from the buyer and deducted from the amount received by the owner.
Fee UTxO	Value	Must be sufficient for all orders spent.

11. Backward Compatibility

- Do not remove or alter existing fields or field types when existing APIs are extended.
- Only add new fields and support both legacy and new behavior simultaneously.
- Do not alter the meaning of an existing API field.